ChatGPT

イメージの作成

イメージを作成するには、プロジェクトのルートに新しいファイルを用意する必要があります。

このファイルは拡張子なしで "Dockerfile" という名前にする必要があります。

最低限必要な構成

このファイルには少なくとも次の内容が必要です：

```
FROM php:8.2-apache
```

`FROM` は、どのイメージを元にするかを指定します。 `scratch`：ベースイメージが不要な場合には "scratch" キーワードを使うこともできます。

コマンドとコピー

```
FROM php:8.2-apache
COPY . /var/www/html
RUN apt update && apt upgrade
```

- `COPY` は、どのファイルやフォルダをどこにコピーするかを指定します。
- `RUN` は、コンテナ上でコマンドを実行するための命令です。
- 複数のコマンドを実行したい場合は、`&&` でつなぐか、複数の `RUN` 命令を書くことができます。

より見やすくするために、バックスラッシュ（`\`）で改行をエスケープすることも可能です。

上記の例では、`php:8.2-apache` イメージを使用し、現在のプロジェクト内のすべてのファイルとフォルダ（`.`）を `/var/www/html` にコピーしています。

なお、`ADD` 命令は `COPY` と似ていますが、URL を扱える点が異なります。

ポートと環境変数の指定

デフォルトで公開するポートや、環境変数も定義できます：

```
FROM mariadb
EXPOSE 3306
ENV MARIADB_ROOT_PASSWORD=root
```

- `EXPOSE` はデフォルトで公開するポート番号を指定します。
- `ENV` は環境変数を設定します。
- 複数の環境変数を定義したい場合は、スペースで区切るか複数の `ENV` を使用します。

Docker イメージのビルド

Dockerfile を作成したら、以下のコマンドでイメージを作成できます：

```
docker build -t nomImage:optionnelTag .
```

- `build` はイメージを構築するためのコマンドです。
- `-t` はイメージに名前を付けるオプションです。必要であれば名前の後に「:」でタグを追加することもできます。
- 最後の「.」は、ビルド対象のプロジェクトディレクトリを示しています（ここでは「.」は現在のディレクトリを意味します）。

Docker Compose

これまで見てきたことは、すべて覚えるには大変です。特に、複数のパソコンでこの構成を再現しなければならない場合はなおさらです。そこで登場するのが Docker Compose です。

Docker Compose を使うと、Docker プロジェクトを構成するための手順を 1 つのファイルに記述でき、1 つのコマンドでそのプロジェクト全体を立ち上げることができます。

---

compose.yaml の作成

プロジェクトのルートに "compose.yaml" というファイルを作成します。

Apache の設定

以前作成した Dockerfile を使ってイメージを追加できます。次のように記述します：

```
services:
    web:
        build: .
        container_name: serverApache2
        restart: always
        depends_on:
```

```
            - mysql
        ports:
            - "8080:80"
        volumes:
            - .:/var/www/html
        environment:
            - SUPER_ENVIRONEMENT_VARIABLE=Mon Super Site
```

注意：YAML 形式ではインデントが非常に重要です。インデントの不足や過剰がエラーの原因になります。

この "compose.yaml" ファイルの内容を細かく説明します：

- `services`：プロジェクトで使用する各サービス（イメージ）を列挙します。
- `web`：各サービスは名前付きプロパティで定義されます。ここでは Apache サーバーを "web" と名付けています。
- `build`：使用する Dockerfile の場所を示します（ここでは `.` は compose.yaml と同じディレクトリ）。
- `container_name`：任意でコンテナに名前を付けることができます（名前はユニークである必要があります）。
- `restart`：コンテナが停止したときの再起動ポリシーを設定します。
- `depends_on`：サービスの起動順を指定します（ここでは、`mysql` サービスの後に起動するよう指定）。
- `ports`：VM とコンテナのポート対応を指定します。
- `volumes`：ボリュームを使ったリンクやバインドマウントの設定です。
- `environment`：環境変数をコンテナまたはアプリケーションに渡します（ここでは例として記載）。

MySQL の設定

既存のイメージを使用してサービスを追加することもできます。引き続き `services` セクションに次を追加します：

```
mysql:
    image: mariadb
    container_name: serverMySQL2
    restart: always
    environment:
        - MARIADB_ROOT_PASSWORD=root
    ports:
        - "3308:3306"
    volumes:
        - ./mysql.sql:/docker-entrypoint-initdb.d/mysql.sql
```

いくつかのプロパティはすでに説明しましたが、その他について説明します：

- `mysql`：任意のサービス名です。
- `image`：使用するイメージとそのタグを指定します。

- `environment`：ここでは実際に機能する環境変数を指定しています。MySQL の初回インストール時にも登場しました。
- `/docker-entrypoint-initdb.d/`：このディレクトリにある SQL ファイルは、コンテナ作成時に自動でデータベースにインポートされます。

PHPMyAdmin の設定

さらに `services` に次のように追記して、PHPMyAdmin を追加します：

```
phpmyadmin:
    image: phpmyadmin
    container_name: serverPHPMyAdmin2
    restart: always
    depends_on:
        - mysql
    environment:
        PMA_ARBITRARY: 1
        PMA_HOST: mysql
    ports:
        - "8081:80"
```

ここでは新しい項目は少ないですが、以下の環境変数に注目してください：

- `PMA_ARBITRARY`：true の場合、MySQL サーバーを選択するフィールドが表示されます。
- `PMA_HOST`：接続するデータベースのホスト名を指定します。

もちろん他にも多くの設定が可能ですが、それについてはご自身で調べてみてください。

Docker Compose のコマンド

ファイルが作成できたら、いよいよプロジェクトの立ち上げです。`compose.yaml` があるプロジェクトのルートディレクトリで以下のコマンドを実行します：

```
docker compose up
```

このコマンドは 3 つのイメージを含むネットワークを作成します。最低限これだけで構いませんが、オプションを加えてターミナルを解放する方法も紹介します。まずコンテナを停止・削除します：

```
docker compose stop
docker compose rm
```

その後、次のように再実行できます：

```
docker compose -p cours-docker up -d
```

- `-p`：Compose プロジェクトに名前を付けます。省略した場合はカレントディレクトリ名が使われます。
- `-d`：バックグラウンドで起動し、ターミナルを占有しません。

Compose に名前を付けた場合、停止や削除もその名前を使って行う必要があります。

Compose の停止と削除をまとめて行うには次のコマンドを使います：

```
docker compose down
```